

PixelCanvas

Bringing Real Pictures to the
World's Oldest Computer Interface

Elliott Humes

Independent Developer

Developed with Anthropic Claude Opus 4.6

February 2026

An accessible overview of `ratatui-pixelcanvas`,
a new open-source tool that lets programmers create
smooth animations and beautiful graphics inside terminal windows.

What Is a Terminal?

If you've ever seen a movie where a hacker types green text on a black screen, you've seen a terminal. Terminals are the oldest way humans talk to computers—older than the mouse, older than the desktop, older than the web browser. They date back to the 1960s, when computers filled entire rooms and people communicated with them by typing commands and reading text responses.

You might think terminals are ancient history, but they're not. Millions of software developers, system administrators, data scientists, and engineers use terminals every single day. When a programmer deploys a website, manages a cloud server, or runs a data analysis script, they're very likely doing it inside a terminal window.

The catch? For over fifty years, terminals have been limited to **text**. Every character sits inside a fixed grid of cells, like a spreadsheet where each cell can hold exactly one letter. You can make text bold, add color, even draw crude boxes using special characters—but you can't show a photograph, draw a smooth curve, or animate anything. It's like having a typewriter in an age of touchscreens.

Until now.

What Is PixelCanvas?

PixelCanvas (formally `ratatui-pixelcanvas`) is a new software tool that lets programmers draw real, high-quality graphics—smooth circles, colorful gradients, animated charts, even 3D rotating objects—directly inside a terminal window. No web browser needed. No separate window. The graphics appear right alongside the text, as if the terminal suddenly learned how to paint.

Think of it this way: before PixelCanvas, building a visual dashboard in a terminal was like trying to create art with LEGO bricks. You could make recognizable shapes, but everything was blocky, coarse, and static. PixelCanvas gives you actual paint and brushes. The results look like something from a modern app or website, but they run inside that same text window programmers have been using for decades.

The tool is written in **Rust**, a programming language known for being extremely fast and reliable—the same language used to build parts of Firefox, Dropbox, and Discord. It's **open-source**, meaning anyone can use it, study it, or improve it for free.

Why Does This Matter?

The Problem It Solves

Imagine you're a data scientist working on a remote server—perhaps analyzing climate data on a supercomputer in another state, or monitoring a fleet of delivery trucks from a coffee shop. You're connected to that server through a terminal over the internet. You have a question: *What does this data look like?*

Before PixelCanvas, you had limited options:

- **Generate an image file**, download it to your laptop, and open it separately. This is slow and breaks your workflow.
- **Use text-based charts** made of ASCII characters (*, |, -). These are fast but ugly and imprecise—imagine trying to read a stock chart made of asterisks.
- **Set up a remote desktop connection** just to see a graph. This is heavyweight and often impractical.

With PixelCanvas, the chart appears *instantly, right in your terminal*. Smooth lines with anti-aliasing. Proper colors. Legends and labels you can actually read. And if the data is updating in real time? The chart *animates smoothly*, updating thirty to sixty times per second—as fluid as a video.

Who Benefits?

- **Software developers** who build command-line tools can now include beautiful, informative visuals in their applications without requiring users to install a graphical interface.
- **Data scientists** can explore datasets visually without leaving their workflow. Scatter plots, histograms, heatmaps—all rendered at publication quality inside the terminal.
- **System administrators** monitoring servers can have animated dashboards showing CPU usage, network traffic, and error rates as smooth, real-time graphics rather than scrolling numbers.
- **Embedded systems engineers** working on devices without screens (routers, IoT sensors, industrial controllers) can now create rich visual interfaces accessible via any terminal connection.
- **AI and machine learning engineers** can visualize training progress, model architectures, and evaluation metrics directly in the terminal where they run their experiments.

How It Works (Without the Jargon)

Think of PixelCanvas as a three-step assembly line:

Step 1: Describe What You Want to Draw

The programmer describes the picture using simple building blocks: “draw a blue circle here,” “fill this rectangle with a gradient from red to orange,” “draw a line graph of these data points.” This description is lightweight—it’s like writing a recipe, not baking the cake yet.

Step 2: Turn the Description Into Pixels

PixelCanvas takes the recipe and “bakes” it—it calculates exactly what color every pixel should be. It uses the same kind of math that powers the graphics in web browsers and mobile apps. Smooth curves are truly smooth, not jagged. Colors blend naturally. Transparency works correctly.

To keep this fast, PixelCanvas remembers what it drew last time. If only one part of the picture changed (say, one chart in a dashboard updated), it only redraws that part. This is like a painter who only repaints the corner of a canvas that needs updating, rather than starting over from scratch.

Step 3: Send the Picture to the Terminal

This is the hardest part, and where PixelCanvas’s innovations really shine. The terminal doesn’t natively understand pictures—it understands text. So PixelCanvas has to encode the picture into a special format the terminal can decode and display.

The clever part: PixelCanvas offers multiple ways to do this, automatically choosing the best one for your terminal:

- **The fast way:** Compress the picture data and stream it efficiently. Like zipping a file before emailing it.
- **The fastest way:** If you’re working locally (not over the internet), PixelCanvas can share the picture through your computer’s memory directly, bypassing the encoding step entirely. This is like handing someone a photograph instead of describing it over the phone.
- **The compatible way:** For older terminals that don’t support fancy protocols, PixelCanvas falls back gracefully, using special Unicode characters to approximate the image—not as pretty, but it always works.

All of these happen automatically. The programmer doesn't need to worry about which terminal someone is using.

What Makes This a Breakthrough?

Modern terminals have technically been able to display pictures since 2017, when the Kitty terminal emulator introduced its graphics protocol. But displaying a *single static image* is very different from running *smooth, real-time animation*. The analogy is the difference between taking a photograph and filming a movie: the fundamental technology exists, but doing it well at 60 frames per second requires solving an entirely different set of engineering challenges.

Before PixelCanvas, no one had solved all of these challenges together:

1. **Speed:** A full-screen picture at modern resolution contains over four million color values. Sending that sixty times per second means moving nearly *a billion bytes per second* through what is essentially a text channel. PixelCanvas uses compression and smart encoding to reduce this by 10–40×.
2. **Smoothness:** Earlier attempts at animation would flicker visibly as each frame was displayed. PixelCanvas coordinates with the terminal to display new frames atomically—the old picture is replaced by the new one in a single instant, with zero visual gap.
3. **Efficiency:** Sending an entire picture every frame is wasteful when 95% of it hasn't changed. PixelCanvas divides the image into a grid of small tiles and tracks which ones actually changed, sending only the modified tiles. For a dashboard where one chart updates, this reduces the amount of data sent by 95%.
4. **Memory:** A naïve approach would create and throw away multiple large memory buffers every frame—imagine printing sixty photographs per second and immediately shredding each one. PixelCanvas reuses the same memory buffers across frames, eliminating this waste entirely.
5. **Compatibility:** Rather than working with only one terminal, PixelCanvas automatically detects which terminal the user is running and adapts its strategy. It works with Kitty, WezTerm, Ghostty, iTerm2, and even basic terminals that don't support graphics at all.

PixelChart: Data Visualization Made Simple

Alongside the core graphics engine, PixelCanvas includes a companion tool called **PixelChart**—a charting library that makes it easy to create common data visualizations:

- **Scatter plots** — for showing relationships between two variables
- **Line charts** — for showing trends over time
- **Bar charts** — for comparing categories (grouped and stacked)
- **Histograms** — for showing data distributions
- **Box plots** — for statistical summaries with outliers
- **Heatmaps** — for showing intensity across two dimensions

These charts include proper axes, legends, labels, and themes—the same elements you’d expect from a professional visualization tool like Excel or Tableau, but running entirely inside a terminal window.

The Role of AI in Development

PixelCanvas was developed through a collaboration between its creator, **Elliott Humes**, and **Anthropic’s Claude Opus 4.6**—one of the most advanced AI coding assistants available. This partnership represents a new model of software development:

- **Elliott** provided the vision, architectural direction, and quality standards—deciding *what* to build and *how well* it should work.
- **Claude** contributed implementation velocity, consistency, and deep technical knowledge—translating high-level goals into working code at a speed that would be difficult for most individual developers.

The result is a codebase of approximately 15,000 lines of professionally structured Rust code, developed in a fraction of the time traditional development would require. This human–AI collaboration model is itself noteworthy: it suggests that complex, performance-critical software can be built effectively through AI partnership, with the human serving as architect and the AI as a highly capable implementer.

A Brief History: How We Got Here

To appreciate what PixelCanvas achieves, it helps to understand just how old the terminal concept is:

1963 — Teletype Model 33

The ancestor of all terminals. A mechanical device that typed computer output onto paper. No screen at all.

1978 — DEC VT100

The terminal that standardized the control codes still used today. Introduced the 80×24 character grid that remains the default terminal size nearly fifty years later.

1983 — DEC VT240 and Sixel

The first attempt at terminal graphics. Images were encoded as columns of six pixels—clever for 1983, but crude by any modern standard. This protocol was nearly forgotten for thirty years.

2013 — iTerm2 Inline Images

Apple's popular Mac terminal added the ability to embed images. A step forward, but designed for static pictures, not animation.

2017 — Kitty Graphics Protocol

A breakthrough: the Kitty terminal introduced a modern protocol with image management, transparency, multiple formats, and shared memory. This gave terminals the *capability* for real graphics—but no one had built the software to fully exploit it for animation.

2024–2025 — Protocol Adoption

WezTerm and Ghostty adopt the Kitty protocol, creating a critical mass of compatible terminals across platforms.

2026 — PixelCanvas

The first library to demonstrate that these capabilities can be used for sustained, smooth, real-time animation—not just static image display.

What Could This Lead To?

PixelCanvas is in its early stages, but the implications extend well beyond drawing circles in a terminal:

Remote Work Visualization

As more technical work happens on remote servers—cloud computing, AI training, scientific simulation—the ability to visualize results *where the work is happening* becomes increasingly valuable. PixelCanvas means a researcher in Tokyo can see beautiful, animated visualizations of a simulation running on a server in Virginia, using nothing more than an SSH connection and a modern terminal.

AI-Powered Dashboards

AI coding assistants are becoming standard tools for developers. PixelCanvas enables these assistants to *show*, not just *tell*. Instead of describing data in text, an AI assistant could generate a chart that appears instantly in the developer's terminal. This creates a richer, more intuitive collaboration between humans and AI.

Accessible Computing

Not every computing environment has (or needs) a full graphical desktop. Embedded devices, servers, containers, and minimal Linux installations often provide only a terminal. PixelCanvas brings rich visual capabilities to these environments without requiring additional infrastructure.

Rethinking What a Terminal Can Be

Perhaps most importantly, PixelCanvas challenges a fifty-year-old assumption: that terminals are fundamentally text-only. By proving that smooth, animated, pixel-perfect graphics are achievable within terminal constraints, it opens the door for a new generation of tools that blend the efficiency and universality of the command line with the visual richness of modern graphical interfaces.

Conclusion

The terminal is one of computing's great survivors. While operating systems, programming languages, and user interface paradigms have come and gone, the terminal has endured for over sixty years—adapting, evolving, and remaining essential to the people who build and maintain the world's software infrastructure.

PixelCanvas represents the next step in that evolution. By solving the engineering challenges that stood between terminal graphics protocols and real-time animation, it transforms the terminal from a text-only interface into a canvas capable of rich, fluid, pixel-perfect visual expression.

The project is open-source, freely available, and still in its early days. Its architecture is designed for extensibility—new graphics protocols, new chart types, and new integration points can be added without disrupting existing functionality. As terminals continue to evolve and as AI tools become more visual, PixelCanvas provides a foundation for a future where the oldest interface in computing is also one of the most visually capable.

Project: <https://github.com/Slush97/ratatui-pixelcanvas>

License: MIT and Apache 2.0 (free to use, modify, and distribute)

Language: Rust

Status: Active development, version 0.1.0